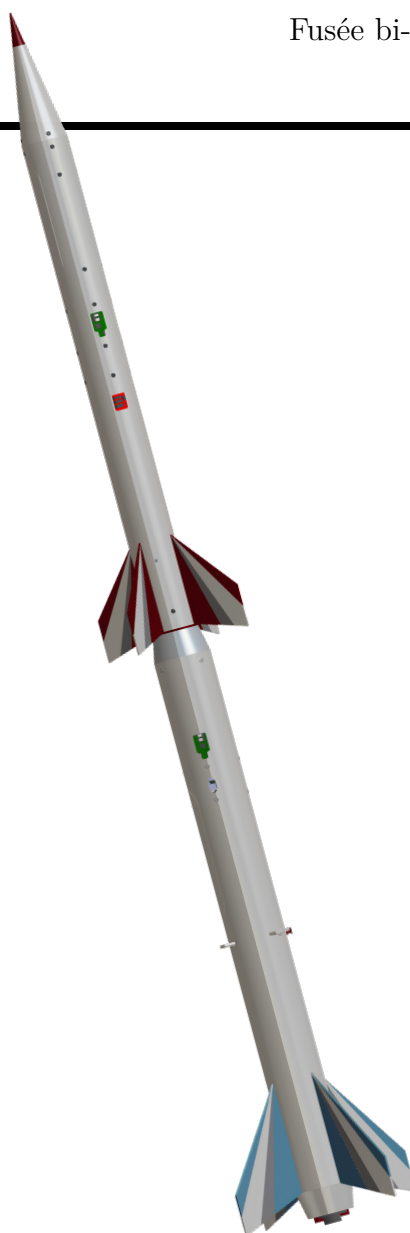




AéroIPSA
63 bvd de Brandbourg
94200 Ivry-sur-Seine

SP - 02

Fusée bi-étage AéroIPSA 2025-2026
Rapport de projet



Rédacteurs du document :

- AMARANTI Malo
- PAILLARD Alexis
- PICHON Lucas
- LE CAM Maxens

Août 2026

Table des matières

1	Logiciel de bord	3
1.1	Méthodologie de développement logiciel embarqué STM32	3
1.1.1	Le framework STM32Cube et la couche d'abstraction matérielle (HAL)	3
1.1.2	Organisation du dépôt : Core , UserProjects et UserLibraries	6
1.1.3	Boucle séquentielle et système d'exploitation temps réel	8
1.2	Séquenceur	9
1.2.1	Architecture générale du programme de vol	9
1.2.2	Sous-programmes	11
1.2.2.1	Fonctions sol	11
1.2.2.2	Phase d'initialisation	13
1.2.2.3	Phase de vol	18
1.2.3	Bibliothèques logicielles du séquenceur	19
1.2.3.1	Contrôle des actionneurs	19
1.2.3.2	Calcul d'attitude	21
1.3	Expérience	25
1.3.1	Présentation générale	25
1.3.2	Real-Time Operating System (RTOS)	25
1.3.3	Gestion des flux de données	25
1.3.4	Chaine de télémessure	25
1.3.4.1	Méthode du Time Division Multiplexing	25
1.3.4.2	Configuration du protocole LORA	25
1.3.4.3	Configuration du protocole FSK	25
1.3.5	Protocole de communication avec le séquenceur	25
1.4	Caméra embarquée	26
1.4.1	Séquence de démarrage	26
1.4.2	Développement du code	26
	Liste des figures	28
	Liste des tableaux	28
	Références	29
	Glossaire et acronymes	30

1 Logiciel de bord

Cette section présente les différents logiciel embarqués à bord de la fusée SP-02. Nous y aborderons les logiciel :

1. du séquenceur ;
2. du système APEX ;
3. des caméras embarquées.

1.1 Méthodologie de développement logiciel embarqué STM32

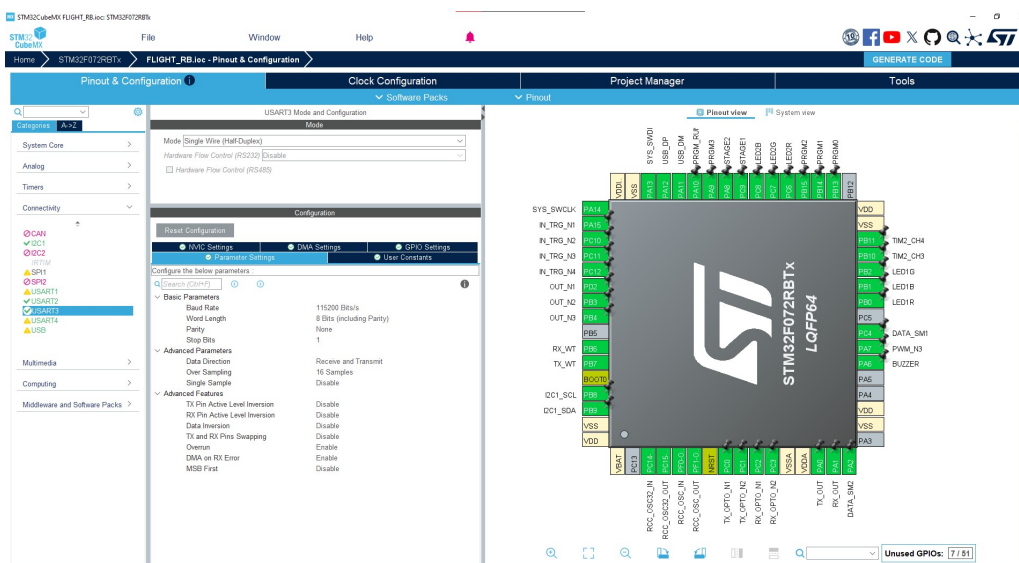
Le séquenceur et l'expérience APEX reposent tous deux sur des microcontrôleurs de la famille STM32, développés au moyen des outils fournis par STMicroelectronics. Avant de détailler le fonctionnement logiciel propre à chacun de ces deux systèmes, cette section présente les principes généraux qui structurent leur développement : l'environnement de génération de code STM32Cube, l'organisation du dépôt logiciel, et les deux paradigmes d'exécution employés sur le projet.

1.1.1 Le framework STM32Cube et la couche d'abstraction matérielle (HAL)

Bien que les membres de l'association soient globalement plus familiers de la programmation Arduino et de l'utilisation de pilotes (*drivers*) de capteurs multiplateformes disponibles en ligne, il a été décidé de privilégier STM32Cube et la HAL, avec des pilotes développés en interne plutôt que des bibliothèques tierces.

Là où l'écosystème Arduino repose sur des cartes de développement prêtes à l'emploi, associées à un environnement qui prend en charge l'essentiel de la configuration du microcontrôleur, l'approche STM32 part du composant lui-même. STMicroelectronics propose une large gamme de microcontrôleurs, déclinés en séries aux ressources et aux performances très variées, et met à disposition un ensemble d'outils logiciels, regroupés sous le nom de STM32Cube, destinés à les configurer et à les programmer. Cet ensemble comprend notamment un outil de configuration graphique, STM32CubeMX, et un ensemble de bibliothèques d'abstraction du matériel, dont la Hardware Abstraction Layer (HAL).

STM32CubeMX permet de définir, pour un microcontrôleur donné, l'affectation des broches, l'arborescence d'horloges ainsi que la configuration des périphériques internes (minuteries, liaisons UART, bus I2C, etc.), puis de générer automatiquement le code d'initialisation correspondant. Ce code prend la forme de fonctions dédiées, une par périphérique configuré (par exemple `MX_USART1_UART_Init`), appelées séquentiellement au démarrage du programme.



Par ailleurs, un microcontrôleur STM32 ne dispose pas d'une fréquence de fonctionnement unique et figée. A partir d'une source primaire (oscillateur interne ou quartz externe), un ensemble de multiplicateurs et de diviseurs permet d'établir la fréquence du cœur, puis celles des différents bus internes auxquels sont rattachés les périphériques. C'est de cette arborescence que découlent, entre autres, la période des minuteries, la vitesse des liaisons série et celle des bus de communication. Un réglage inadapté ne se traduit généralement pas par une erreur de compilation, mais par un comportement temporel erroné à l'exécution, ce qui rend sa vérification d'autant plus importante. STM32CubeMX en propose une représentation graphique complète, dans laquelle chaque fréquence intermédiaire est affichée et contrôlée vis-à-vis des limites du composant.

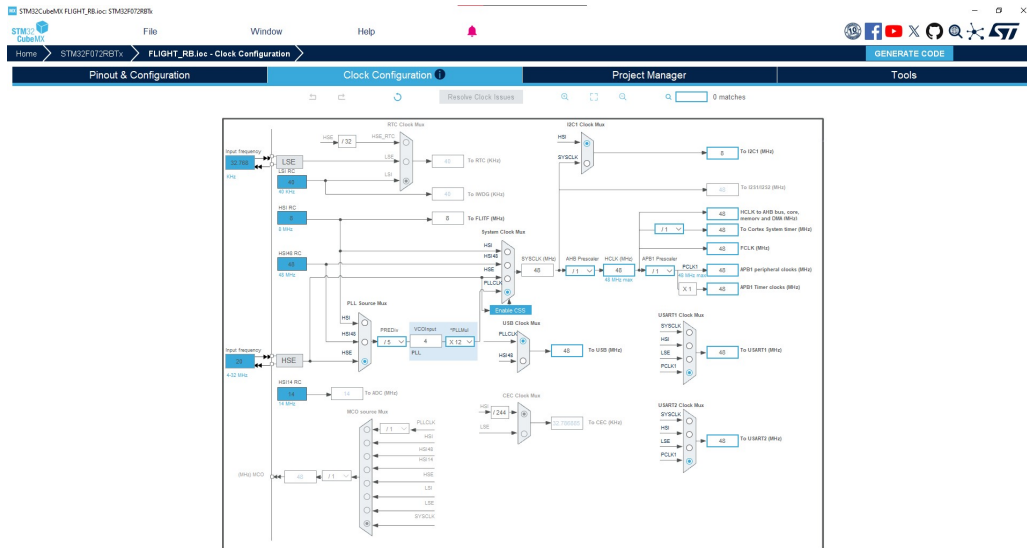


FIGURE 1.2 – Configuration d'horloges avec STM32CubeMX

Cette étape de configuration n'a pas d'équivalent dans le monde Arduino, où elle est réalisée implicitement par le framework. L'abstraction y est poussée au point de masquer largement l'architecture sous-jacente : un même programme peut être compilé presque à l'identique pour une carte à base d'AVR ou pour un ESP32. Sur STM32, la cible compte réellement, et son paramétrage représente un travail supplémentaire, en contrepartie duquel l'intégralité du comportement du composant reste visible et modifiable.

Le code d'initialisation produit par STM32CubeMX s'appuie quant à lui sur la Hardware Abstraction Layer (HAL), une bibliothèque logicielle qui expose une interface de programmation (API) unifiée pour piloter les périphériques du microcontrôleur (HAL_GPIO_WritePin, HAL_UART_Transmit, HAL_I2C_Master_Receive, etc.), sans que le développeur ait à manipuler directement les registres internes du composant. Son rôle est comparable à celui que joue le framework Arduino avec des fonctions telles que `digitalWrite` ou `Serial.print`, mais elle se situe à un niveau nettement plus proche du matériel : chaque fonction agit sur un périphérique explicitement identifié et préalablement configuré, et son effet sur les registres reste directement traçable dans le code source fourni. Cette proximité se traduit également par un code exécutable plus léger et plus prévisible : là où une écriture sur une broche traverse, côté Arduino, plusieurs tables de correspondance et vérifications avant d'atteindre le registre concerné, l'équivalent HAL se réduit à quelques instructions. Le surcoût ne provient pas du compilateur, identique dans les deux cas, mais du nombre de couches d'abstraction traversées à l'exécution. Dans un domaine aussi exigeant que l'aérospatial, pouvoir estimer ce que devient réellement une ligne de code une fois compilée constitue un critère de choix à part entière.

L'API unifiée de la HAL permet par ailleurs de transposer un même code applicatif d'une série STM32 à une autre. Cette transposition reste toutefois conditionnée au maintien d'une configuration matérielle explicite, propre à chaque cible : la HAL masque les registres, pas le microcontrôleur. Les pilotes développés en interne viennent se greffer au-dessus de cette couche : ce sont eux qui portent la logique propre à chaque composant externe, en s'appuyant sur les fonctions de la HAL pour les échanges bas niveau.

L'écosystème Arduino (Arduino Nano, ESP32, framework Arduino) répond quant à lui à des besoins différents, et conserve des atouts réels pour un projet associatif : une prise en main rapide, une abondance de bibliothèques déjà écrites pour la quasi-totalité des capteurs et actionneurs du commerce, et donc un temps de développement initial très court. Piloter un servomoteur ou une centrale inertielle s'y résume souvent à quelques appels de fonction. Ces qualités ont cependant une contrepartie : ces bibliothèques sont conçues pour un usage générique, sans garantie sur leur comportement temporel (fonctions bloquantes non documentées, boucles d'attente actives...) ni sur la qualité de leur implémentation, et leur fonctionnement interne reste, pour l'essentiel, une boîte noire pour qui les utilise. Le constat s'étend au-delà du seul comportement temporel : la configuration des ressources partagées entre plusieurs composants y est généralement figée par défaut et jamais exposée au développeur, de sorte que dès que plusieurs composants aux besoins différents doivent cohabiter sur une même ressource (un bus de données SPI par exemple), ce réglage implicite devient difficile à ajuster, voire à diagnostiquer en cas de conflit. Ces inconnues sont difficilement compatibles avec un système impliquant la sécurité du vol, où le comportement de chaque fonction exécutée doit pouvoir être vérifié.

Le choix de pilotes « maison » répond à ces limites. Chaque pilote (servomoteurs STS, centrale inertielle WT901B, LED, buzzer...) et chaque utilitaire logiciel (tampon circulaire, messagerie interne par abonnement, planification d'animations LED...) a été écrit au sein de l'équipe, au-dessus de la HAL. Aucune ligne de code exécutée en vol ne provient ainsi d'une source dont le comportement n'a pas été établi, ce qui rend possible l'audit du budget temporel de chaque fonction appelée.

Ce choix répond également à une volonté de développement de compétences techniques personnelles. Configurer soi-même sa cible oblige à en comprendre le fonctionnement interne, et l'écriture puis la validation de ses propres pilotes constituent, dans le même esprit, un objectif d'apprentissage à part entière en programmation comme en électronique numérique bas niveau, complémentaire à la réalisation du projet lui-même.

Cette approche a toutefois un coût. Le développement de chaque pilote et de chaque utilitaire depuis zéro demande un temps bien supérieur à l'intégration d'une bibliothèque existante, sans bénéficier de la robustesse qu'apporte un usage et une correction communautaires étalés dans le temps : chaque bogue rencontré doit être identifié et corrigé en interne, sans retour d'expérience extérieur. Elle alourdit également l'intégration de nouveaux membres au sein de l'association, moins familiers d'un développement bas niveau que du monde Arduino, et fait reposer l'intégralité de la maintenance du code sur quelques développeurs.

1.1.2 Organisation du dépôt : Core, UserProjects et UserLibraries

Le développement de plusieurs cartes distinctes, chacune donnant lieu à un programme de vol et à une série de programmes de test, impose une organisation de dépôt répondant à trois besoins :

- Transposer facilement une bibliothèque écrite en interne d'un projet STM32Cube à un autre, sans avoir à la réécrire ni à l'extraire manuellement de son contexte d'origine. Par exemple, le séquenceur et l'expérience APEX utilisent tous deux le capteur WT901B et donc disposent d'un driver commun.
- Assurer la traçabilité de l'ensemble des tests menés au fil du développement itératif du code : chaque programme de test écrit à une étape donnée reste présent dans le dépôt et reste exécutable par la suite.
- Pouvoir passer très rapidement d'un programme à un autre, afin de conduire des tests de non-régression sans manipulation lourde.

L'arborescence retenue, commune aux deux systèmes STM32 du projet, repose sur trois répertoires principaux : **Core**, **UserProjects** et **UserLibraries**.

Core – Le répertoire **Core** contient le code généré par STM32CubeMX : initialisation des horloges et des périphériques, gestionnaires d'interruption, ainsi qu'un fichier **main.c** volontairement minimal. Outre les en-têtes générés décrivant les périphériques configurés (**main.h**, **gpio.h**, **tim.h**, **usart.h**, **i2c.h**, **dma.h**, ...), ce dernier n'inclut qu'un seul en-tête applicatif, **project.h**, qui déclare deux fonctions : **setup()**, appelée une fois le matériel initialisé, et **loop()**, appelée à chaque itération de la boucle infinie qui suit.

Ce « main générique » ne contient aucune logique applicative. Il constitue un point d'entrée neutre, identique quel que soit le programme réellement exécuté par la carte, et n'a plus à être modifié une fois la configuration matérielle établie. La structure **setup()/loop()** reprend volontairement celle du framework Arduino, familière à l'ensemble des membres de l'association.

```
#include "main.h"
#include "gpio.h"
...

#include "project.h"

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_I2C1_Init();
    ...

    setup();
    while (1) {
        loop();
    }
}
```

```
// project.c
```

```
void setup() {
    ...
}

void loop() {
    ...
}
```

UserProjects – Le fichier **project.h** inclus par le **main** et son implémentation **project.c** appartiennent au répertoire **UserProjects**. Celui-ci regroupe, sous forme d'un sous-dossier par programme, l'ensemble des firmwares développés au cours du projet : le programme de vol (**full_seq**), ses variantes par étage (**full_seq_stage_1**, **full_seq_stage_2**), ainsi qu'une série de programmes de test dédiés à la validation isolée d'un sous-système (**test_servo**, **test_uart**, **test_wt901b**, **test_ihf**, **test_buzzer**, etc.). Chaque sous-dossier possède ainsi ses propres **project.h** et **project.c**, et donc sa propre implémentation de **setup()** et **loop()**.

Cette organisation sert directement les deux derniers besoins évoqués plus haut. Plutôt que d'intégrer d'emblée l'ensemble des sous-systèmes (servomoteurs, centrale inertielle, mise à feu, interface homme-fusée...) dans un unique programme complexe, chacun a pu être développé et éprouvé indépendamment dans un firmware dédié, réduisant d'autant la surface d'erreur au moment de l'intégration finale. Ces programmes n'étant jamais supprimés, ils constituent un historique exécutable des validations menées : en cas de dysfonctionnement ou après une modification d'une bibliothèque partagée, il reste possible de reprendre un sous-système isolément sans avoir à interpréter le comportement du programme complet.

UserLibraries – Le troisième répertoire, **UserLibraries**, rassemble le code partagé entre ces programmes. Il se divise lui-même en deux familles. Le sous-répertoire **drivers** contient les pilotes propres à un composant matériel donné (STS pour les servomoteurs, WT901B pour la centrale inertielle, led, buzzer), tandis que **utils** regroupe les utilitaires indépendants du matériel (**circular_buffer**, **data_topic**, **led_scheduler**, **actuator**, **quaternion**, **iir_filter**, **waveform**...). Chaque bibliothèque étant autonome et rassemblée dans un dossier unique, elle peut être employée par n'importe quel programme du dépôt, et transposée telle quelle vers un autre projet STM32Cube en recopiant ce seul dossier et en déclarant ses chemins dans la configuration de compilation. Une correction apportée à une bibliothèque bénéficie par ailleurs immédiatement à l'ensemble des firmwares qui l'utilisent.

Chaque module, qu'il s'agisse d'une bibliothèque ou d'un projet, suit la même arborescence interne : un dossier **Core/Inc** pour les en-têtes et un dossier **Core/Src** pour les sources. Les premiers sont déclarés comme chemins d'inclusion et les seconds comme fichiers à compiler, de sorte que n'importe quel fichier du projet peut inclure un en-tête par son seul nom, sans avoir à exprimer un chemin relatif dépendant de sa propre position dans l'arborescence.

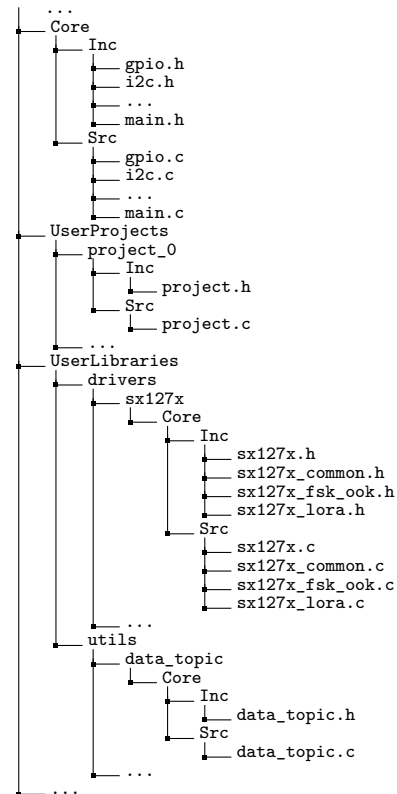


FIGURE 1.3 – Arborescence d'un dépôt logiciel type utilisé pour le séquenceur et l'expérience APEX

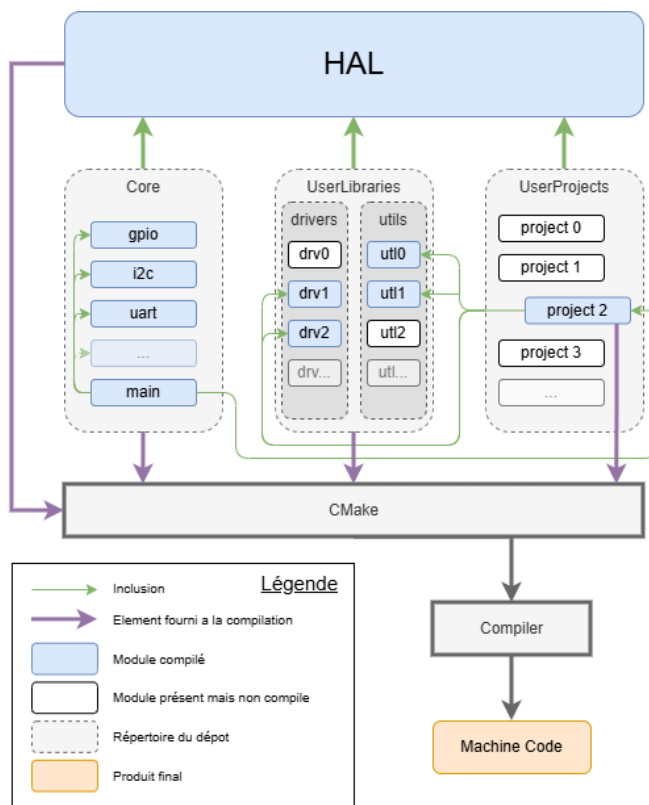


FIGURE 1.4 – Chaîne de compilation d'un dépôt logiciel type du projet

La sélection du programme à compiler s'opère enfin au niveau de la configuration CMake, par l'intermédiaire d'une variable **CMAKE_USER_PROJECT_PATH** désignant le sous-dossier de **UserProjects** retenu. Cette variable est ensuite utilisée pour compléter la liste des chemins d'inclusion et celle des fichiers sources, si bien que l'ajout d'un nouveau programme ne demande que la création d'un sous-dossier respectant la convention **Core/Inc** et **Core/Src**. Les autres chemins restent présents dans le fichier de configuration sous forme de lignes commentées : basculer d'un programme à l'autre revient à déplacer un commentaire, sans modification du reste du dépôt.

1.1.3 Boucle séquentielle et système d'exploitation temps réel

Au-delà de cette organisation commune, le séquenceur et l'expérience APEX diffèrent sur un point structurant : le modèle d'exécution retenu pour la boucle `loop()`.



FIGURE 1.5 – Boucle séquentielle (superloop)

Le séquenceur repose sur une architecture dite en boucle séquentielle (ou superloop) : à chaque itération, `loop()` exécute successivement une série de fonctions « pas à pas », chacune non bloquante, qui font avancer l'état du système d'une petite quantité avant de rendre la main. Il n'existe qu'un seul flux d'exécution, sans ordonnanceur ni préemption : le programme fait exactement ce que son code décrit, dans l'ordre où il le décrit. Cette simplicité a une contrepartie stricte : aucune fonction appelée dans la boucle ne doit bloquer plus de quelques centaines de microsecondes, sous peine de retarder l'ensemble des autres tâches du système. Le budget de temps d'exécution de chaque fonction doit donc être audité individuellement.

Cette contrainte a une conséquence structurelle sur la manière d'écrire le code. Une tâche qui, écrite naturellement, s'exprimerait de façon linéaire (par exemple envoyer une commande, attendre une réponse, traiter la réponse) doit être découpée en étapes élémentaires réparties sur plusieurs itérations de la boucle. La majeure partie du temps chaque tâche doit être réécrite sous forme de machine à états, et tout ce qui constituerait normalement le contexte local de la fonction doit être conservé explicitement dans des variables d'état persistantes entre les appels.

Tant que les tâches sont peu nombreuses et simples, ce coût reste modeste. Il croît en revanche rapidement avec le nombre de tâches concurrentes et le nombre d'étapes de chacune : la liste des variables à maintenir s'allonge, et les machines à états deviennent difficiles à faire évoluer sans risque de régression. C'est précisément lorsque cette complexité devient prépondérante que le recours à un système d'exploitation temps réel prend tout son sens : celui-ci fournit nativement les mécanismes de sauvegarde et de restauration de contexte que la boucle séquentielle impose d'écrire à la main.

L'expérience APEX repose quant à elle sur un système d'exploitation temps réel (RTOS), en l'occurrence FreeRTOS. Le programme y est découpé en plusieurs tâches, ordonnancées de manière préemptive par le noyau de l'OS, chacune pouvant se mettre en attente (sur un délai, une donnée à recevoir, une ressource partagée) sans bloquer l'exécution des autres tâches. Chaque tâche disposant de sa propre pile, son contexte est sauvegardé et restauré automatiquement à chaque commutation : le code peut donc rester écrit de manière linéaire, sans machine à états ni variables d'état explicites.

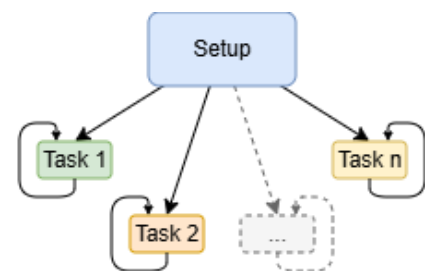


FIGURE 1.6 – Système d'exploitation temps réel (RTOS)

Cette architecture facilite le développement de systèmes devant gérer plusieurs flux simultanés et faiblement synchronisés entre eux, au prix d'une complexité supplémentaire, déplacée ailleurs : protection des ressources partagées entre tâches, dimensionnement de la pile de chacune, et une garantie de déterminisme temporel global plus difficile à établir que sur une boucle séquentielle unique.

Le choix de l'un ou l'autre paradigme sur chaque système découle directement de leurs contraintes respectives : le séquenceur, soumis à des exigences fortes de sûreté de vol et de déterminisme temporel, retient la boucle séquentielle ; l'expérience APEX, qui doit gérer en parallèle l'acquisition de plusieurs capteurs, deux protocoles de télémesure et la communication avec le séquenceur, retient FreeRTOS.

1.2 Séquenceur

1.2.1 Architecture générale du programme de vol

Les deux étages de la fusée embarquent une carte séquenceur identique. Les rôles de ces deux cartes diffèrent pourtant sensiblement : le premier étage commande les aérofreins, la séparation et son propre parachute, tandis que le second commande l'allumage de son propulseur et son parachute. Plutôt que de maintenir deux firmwares distincts, dont la partie commune aurait dû être tenue à jour en double, un unique firmware (`full_seq`) est compilé pour les deux cartes. Cela écarte également tout risque de charger par erreur le programme du second étage sur la carte du premier, ou inversement. L'architecture générale du programme est représentée sur la figure 1.7 ci-après.

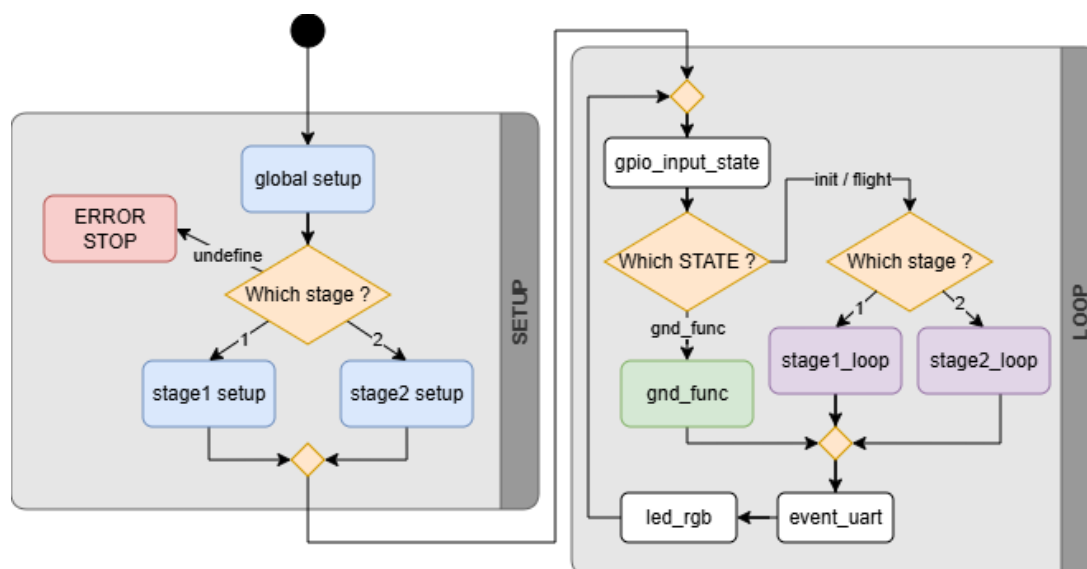


FIGURE 1.7 – Architecture générale du programme du séquenceur

Le programme se décompose en deux temps, visibles sur la figure. La fonction `setup()`, exécutée une seule fois à la mise sous tension, réalise l'initialisation de la carte : configuration du matériel, puis détection de l'étage sur lequel elle est installée, d'où découle l'initialisation des organes qui lui sont propres. La fonction `loop()` prend ensuite le relais et s'exécute indéfiniment : c'est la boucle séquentielle décrite en section ??.

Ce firmware unique ne réalise pas une action unique. Il embarque un ensemble de sous-programmes, correspondant chacun à une opération que la carte doit pouvoir exécuter, dont un seul s'exécute à la fois. C'est la boucle qui, à chaque itération, fait progresser le sous-programme actif. Le choix de ce dernier revient à l'opérateur, au moyen du sélecteur de programme de l'IHF. Le schéma en distingue deux natures :

- Les fonctions sol (*ground functions*), en vert, regroupent les opérations de maintenance et de vérification employées pendant les campagnes de test et lors de l'assemblage. Chacune se termine d'elle-même et rend la main au séquenceur, qui se remet en attente d'une nouvelle consigne.
- Le programme de campagne de lancement, en violet, correspond quant à lui au déroulement d'un lancement : une fois engagé, il ne rend jamais la main et enchaîne l'armement de la fusée puis son vol.

Détection de l'étage Le firmware étant commun aux deux cartes, celles-ci doivent pouvoir déterminer seules le rôle qu'elles ont à tenir. Lors du montage de la fusée, un jumper peut être placé sur le séquenceur selon deux configurations différentes, correspondant à l'étage sur lequel il est installé. Le programme lit cette configuration au démarrage. Si aucune des deux états est détecté, du fait que le jumper n'est pas présent par exemple, la configuration est jugée invalide : la LED passe au rouge et le programme s'arrête, plutôt que de laisser une carte s'exécuter sans savoir quel étage elle pilote.

Cette lecture, effectuée une seule fois, conditionne tout le reste de l'exécution. Elle détermine d'une part la fonction d'initialisation appelée (`setup_stage_1` ou `setup_stage_2`), qui déclare les entrées utilisées par l'étage et initialise ses actionneurs. Elle détermine d'autre part le contenu des fonctions sol : une même position du sélecteur ne désigne pas la même opération selon l'étage, chacun ne disposant pas des mêmes organes.

Structure de la boucle La fonction `loop()` conserve une structure constante tout au long de l'exécution, quel que soit le sous-programme actif. Elle se décompose en quatre étapes, décrites et illustrées ci-après :

```
void loop() {
    update_gpio_input_states (...);

    switch (current_phase_type) {
        case STAGE_PHASE_GROUND_FUNC:
            ...
            break;
        case STAGE_PHASE_INIT:
        case STAGE_PHASE_FLIGHT:
            switch (stage) {
                case ROCKET_FIRST_STAGE: {
                    loop_stage_1 (...);
                    break;
                }
                case ROCKET_SECOND_STAGE: {
                    loop_stage_2 (...);
                    break;
                }
                case ROCKET_STAGE_NO_SET: {
                    // Wait what ???
                }
            }
            break;
    }

    event_uart_producer_run (...);

    LED_RGB_SetColor (...);
}
```

— `update_gpio_input_states` relève l'état de l'ensemble des entrées électronique de la carte. Ce relevé groupé est préférable à une lecture de chaque broche au moment où une condition l'exige : toutes les décisions prises au cours d'une même itération s'appuient ainsi sur une image cohérente des entrées, prise à un instant unique, et une même broche ne peut pas être vue dans deux états différents par deux conditions successives. De plus, cela permet de mettre plus simplement en place un mécanisme de filtrage logiciel des entrées. Bien qu'il ne fut pas nécessaire pour le projet, cette fonctionnalité a été étudiée.

— L'aiguillage vers le sous-programme actif, qui distingue les fonctions sol (`STAGE_PHASE_GROUND_FUNC`) du programme de campagne de lancement. Ce dernier se subdivise en deux phases, la phase d'initialisation en rampe (`STAGE_PHASE_INIT`) et le vol (`STAGE_PHASE_FLIGHT`), Cette distinction reste toutefois sans effet ici, puisque les deux cas conduisent à la même fonction, propre à l'étage, la logique de la phase étant gérée à l'intérieur de cette fonction.

— `event_uart_producer_run` assure la transmission, vers l'expérience APEX, du journal des événements survenus dans le séquenceur. Son fonctionnement est détaillé en section ??.

— `LED_RGB_SetColor` met à jour la LED de signalisation, unique retour d'information disponible sur la fusée une fois celle-ci refermée.

Une contrainte d'écriture La boucle ne doit jamais être bloquée durablement, sous peine de retarder la surveillance des événements de vol. Aucune opération longue ne peut donc y être écrite d'un seul tenant : chaque sous-programme est réalisé sous forme de machine à états, dont l'état est conservé d'une itération à l'autre et qui progresse d'au plus une transition par passage. Cette contrainte ne s'applique en revanche pas à `setup()`, qui s'exécute avant toute échéance temporelle et peut donc attendre la fin d'une initialisation matérielle sans conséquence.

1.2.2 Sous-programmes

La section précédente a décrit la structure du firmware et la manière dont il aiguille l'exécution vers l'un ou l'autre de ses sous-programmes. Celle-ci en détaille le contenu, c'est-à-dire ce que le séquenceur exécute réellement.

Deux types de sous-programmes coexistent dans le firmware :

- Les fonctions sol, d'une part, permettent de manipuler individuellement les organes de la fusée lors des essais et de l'assemblage. Chacune constitue une opération indépendante, déclenchée à la demande et répétable à volonté.
- Le programme de campagne de lancement, d'autre part, couvre le déroulement complet d'un lancement. Il constitue un sous-programme unique, mais se déroule en deux temps : une phase d'initialisation, qui amène la fusée à un état vérifié et prêt au décollage, puis une phase de vol, qui s'exécute sans aucune intervention possible.

1.2.2.1 Fonctions sol

De nombreuses opérations doivent pouvoir être commandées en dehors de tout vol : déployer les aérofreins pour vérifier leur débattement, ouvrir et refermer une trappe de parachute, actionner le système de séparation lors de l'assemblage des deux étages... Le séquenceur reste accessible et reprogrammable même fusée assemblée, mais recompiler et téléverser un firmware pour chacune de ces manipulations serait fastidieux, et surtout risqué : la carte qui vole ne serait plus celle qui a été validée. Ces opérations sont donc embarquées une fois pour toutes dans le firmware, sous forme de fonctions sol, et rendues accessibles par leur numéro depuis l'IHF.

Sélection et exécution En l'absence de fonction sol active, le séquenceur attend un appui sur le bouton poussoir. La position du sélecteur n'est lue qu'à cet instant précis, et non en continu : manipuler le sélecteur ne déclenche donc rien tant que l'opérateur n'a pas validé son choix. La valeur lue détermine la suite : la position 0 engage le programme de campagne de lancement, tandis que les positions 1 à 15 sélectionnent la fonction sol correspondante, qui devient alors la fonction active. Le tableau 1 récapitule les fonctions disponibles sur chaque étage.

Contrat d'une fonction sol Toutes les fonctions sol partagent la même signature et le même contrat. Appelées à chaque itération de la boucle tant qu'elles sont actives, elles renvoient `GROUND_FUNC_STATE_RUNNING` pour signaler qu'elles n'ont pas terminé, ou `GROUND_FUNC_STATE_DONE` pour rendre la main au séquenceur, qui se remet alors en attente d'une nouvelle consigne. Une opération de plusieurs secondes n'est donc jamais exécutée d'un seul tenant : elle conserve son avancement dans des variables persistantes et progresse d'un pas à chaque appel.

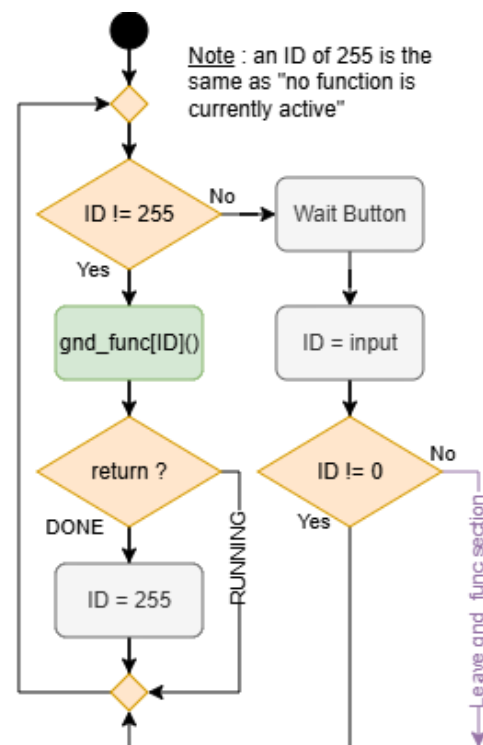


FIGURE 1.8 – Sélection et exécution d'une fonction sol

N°	Nom	Description
<i>Premier étage</i>		
0	Programme de lancement	Déroulement complet d'un lancement
1	Référence aérofrens	Recherche de butée de l'actionneur d'aérofrens
2	Référence trappe	Recherche de butée de la trappe de parachute
3	Référence séparation	Recherche de butée du système de séparation
4	Manœuvre aérofrens	Déplacement d'une position à chaque appui
5	Manœuvre trappe	Déplacement d'une position à chaque appui
6	Manœuvre séparation	Déplacement d'une position à chaque appui
7 à 15	—	Emplacements réservés
<i>Second étage</i>		
0	Programme de lancement	Déroulement complet d'un lancement
1	Référence trappe	Recherche de butée de la trappe de parachute
2	Manœuvre trappe	Déplacement d'une position à chaque appui
3	Validation attitude	Contrôle des conditions d'allumage et de la chaîne associée
4	Test allumage	Activation de la sortie d'allumage tant que la fonction est active
5 à 15	—	Emplacements réservés

TABLE 1 – Fonctions sol disponibles sur chaque étage

Opérations sur les actionneurs Les sous-programmes 1 à 6 pour le premier étage et 1 à 2 pour le second étage commandent des actionneurs. Il existe deux types d'opérations sur ces organes : la mise en référence et la manœuvre :

- La *mise en référence* lance, au premier appel, la recherche d'une butée mécanique d'un actionneur et active un motif lumineux d'attente. Aux appels suivants, elle interroge l'avancement de l'opération et renvoie **RUNNING** tant que celle-ci n'est pas achevée. Dès que la butée est atteinte, c'est à dire que le servomoteur n'arrive plus à évoluer, la fonction renvoie **DONE**.
- La *manœuvre* met en mouvement un actionneur. Ces derniers ont généralement plusieurs positions, correspondant à des angles de rotation différents. La fonction de manœuvre progresse d'une position vers la suivante à chaque appui sur le bouton, en changeant de sens une fois la dernière (ou la première) position atteinte. L'opérateur peut ainsi parcourir l'ensemble des positions d'un organe par appuis successifs, et vérifier visuellement son débattement. La fonction refuse de s'exécuter si l'actionneur n'a pas été préalablement mis en référence, ses positions n'ayant alors aucune signification.

Fonctions liées à l'allumage La position 3 du deuxième étage valide la condition qui autorisera l'allumage du propulseur en vol. Cette fonction acquiert en continu les mesures de la centrale inertielle, en déduit l'élévation et l'azimut de la fusée, puis les compare aux valeurs attendues au moment de l'allumage. Le résultat est restitué par la couleur de la LED :

- verte si les deux angles sont dans leur tolérance ;
- bleue si un seul l'est ;
- rouge sinon.

La sortie d'allumage est pilotée en conséquence, ce qui permet de valider la chaîne complète, depuis la mesure d'attitude jusqu'à la commande. Les grandeurs calculées sont par ailleurs émises sur la liaison USB, afin de pouvoir être tracées et vérifiées finement en atelier. L'opérateur bascule entre les modes (accélération et gyromètre) et quitte la fonction par des appuis courts ou longs sur le bouton. Plus d'informations sur la mesure d'attitude sont disponibles en section ??.

La position 4 est plus directe : elle maintient la sortie d'allumage active tant que la fonction s'exécute, et la désactive au moment d'en sortir. Elle permet de vérifier la continuité de la chaîne pyrotechnique indépendamment de toute condition d'attitude. Ces deux fonctions ne sont évidemment employées qu'avec un dispositif de mesure en lieu et place de l'allumeur.

1.2.2.2 Phase d'initialisation

Sélectionner la position 0 engage le programme de campagne de lancement, qui s'ouvre sur une phase d'initialisation. Son rôle est d'amener la fusée, depuis un état quelconque, jusqu'à un état vérifié et prêt au décollage. Elle se distingue des fonctions sol sur un point essentiel : l'opérateur n'y choisit plus ce qu'il exécute. Le séquenceur impose un ordre, valide chaque étape avant de passer à la suivante, et refuse d'avancer tant que les conditions ne sont pas réunies. La procédure de préparation est ainsi rendue obligatoire par le logiciel, plutôt que confiée à la rigueur de l'opérateur.

Une progression validée pas à pas

Une progression validée pas à pas La phase se déroule comme une succession d'étapes séparées par des attentes d'appui sur le bouton. Ce découpage répond à une règle que s'est fixée l'équipe : aucun servomoteur ne doit jamais être mis en mouvement automatiquement. Toute commande d'actionneur est précédée d'une validation explicite de l'opérateur, qui s'assure au préalable que rien n'entrave la course du système concerné.

Les étapes s'enchaînent dans l'ordre suivant sur le premier étage, représenté sur la figure 1.9 : attente de l'insertion du JACK READY, puis mise en référence successive des aérofreins, de la trappe de parachute et du système de séparation, et enfin vérification globale de l'état de la fusée. Le second étage suit la même trame, réduite aux organes dont il dispose : JACK READY, mise en référence de sa trappe, puis vérification globale (1.11).

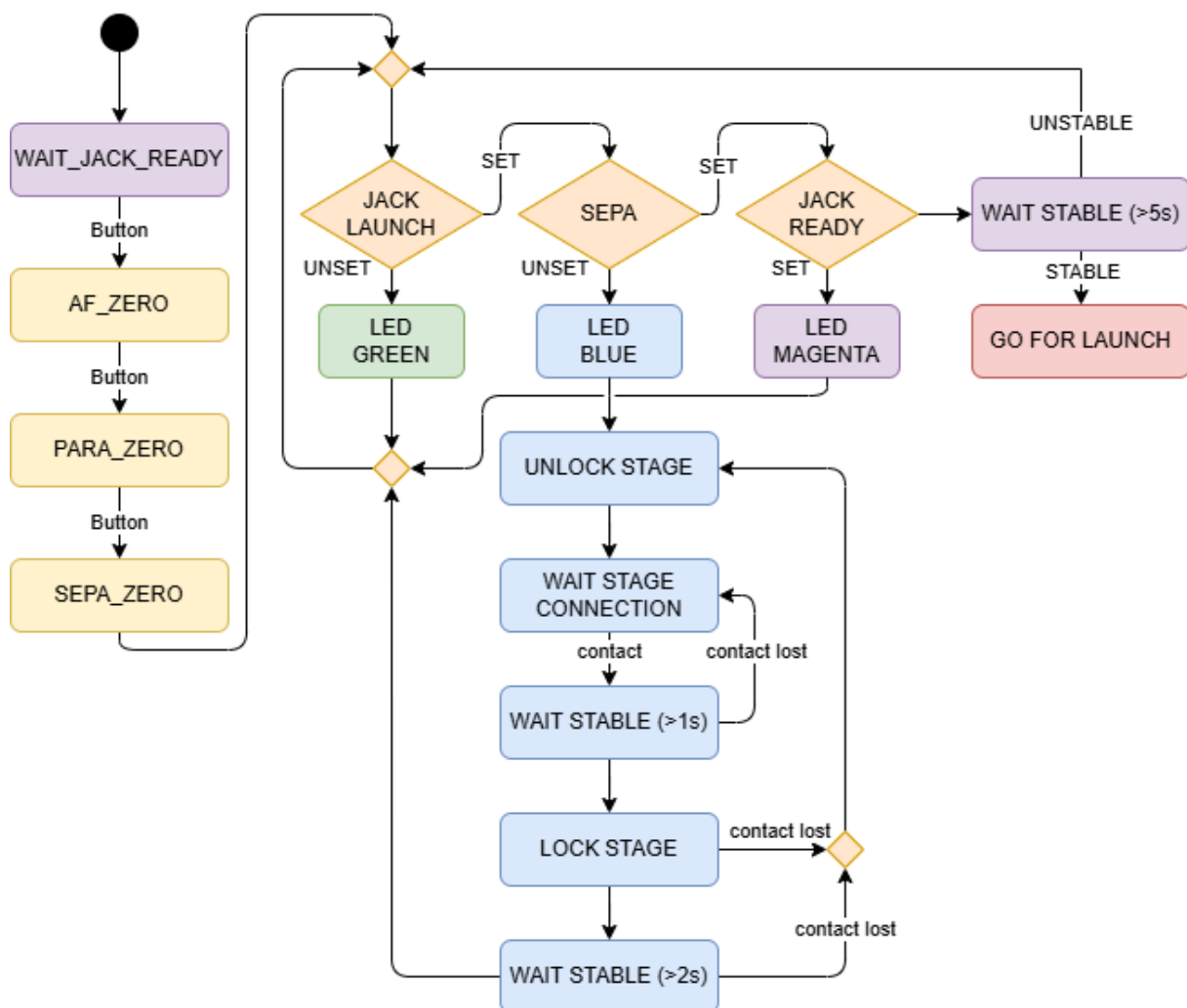


FIGURE 1.9 – Phase d'initialisation du premier étage.

La vérification globale Une fois les actionneurs mis en référence, la fusée entre dans une boucle de surveillance. L'opérateur n'y déclenche plus d'étape : le séquenceur évalue désormais de lui-même les conditions nécessaires au départ, et pilote au passage l'assemblage des étages décrit plus loin. Deux de ces conditions portent sur la configuration mécanique de la fusée : le **JACK_LAUNCH** doit être inséré, et les deux étages doivent être assemblés avec le mécanisme de séparation verrouillé. La troisième, le retrait du **JACK_READY**, n'est évaluée qu'une fois les deux premières satisfaites.

La figure présente ces vérifications sous forme d'une chaîne de tests, mais il faut se garder de la lire comme une progression figée : l'ensemble du cycle est reparcouru à chaque itération de la boucle principale. Les conditions sont donc réévaluées en permanence, et rien n'est acquis une fois pour toutes. Si les étages sont désolidarisés après avoir été assemblés, ou si le **JACK_LAUNCH** est retiré, la condition correspondante redevient insatisfaite et la fusée recule d'autant : c'est l'état courant du système qui compte, et non l'historique des étapes franchies.

Le **JACK_READY** est arraché par les derniers opérateurs quittant le pas de tir, rampe déjà érigée. Tant qu'il est en place, le séquenceur ne peut en aucun cas interpréter un faux contact ou une secousse liée aux manipulations comme un décollage, et donc engager sa séquence de vol, ce qui conduirait, sur le second étage, à l'allumage du propulseur alors que des personnes se trouvent à proximité immédiate. Son retrait constitue par ailleurs le signal de départ de la chaîne de télémessure de l'expérience APEX. C'est pourquoi sa présence est exigée en début de séquence, et son retrait demandé en toute fin.

L'assemblage des étages La partie bleu de la figure détaille la séquence d'assemblage. Le premier étage, seul sur la rampe, déverrouille son mécanisme de séparation puis attend le couplage du second étage. Dès que le contact est détecté et maintenu pendant une seconde, il commande le verrouillage, puis vérifie pendant deux secondes supplémentaires que ce contact reste établi avant de considérer l'assemblage abouti. Toute perte de contact fait reculer la séquence à l'étape correspondante.

L'assemblage est ainsi entièrement automatique : les opérateurs n'ont qu'à emboîter les deux étages, le séquenceur se chargeant du reste sans qu'aucune commande ne lui soit adressée. Si le contact est perdu, alors le séquenceur déverrouille le mécanisme de séparation pour permettre un retour à un état pouvant accueillir le deuxième étage. La double temporisation écarte par ailleurs les détections fugitives, fréquentes lors d'un emboîtement mécanique : une prise de contact intermittente pendant l'ajustement ne déclenche pas de verrouillage prématuré.

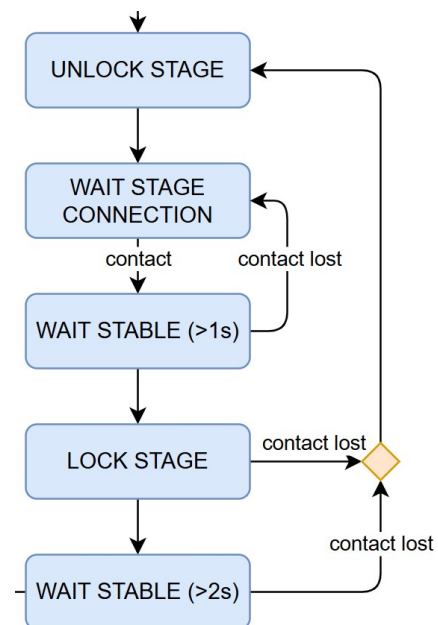


FIGURE 1.10 – Zoom sur la séquence d'assemblage des étages.

Enfin, une action volontaire reste accessible à l'opérateur pendant toute la durée de cet état, bien qu'elle ne figure pas sur la figure 1.9 pour ne pas en alourdir la lecture : la commande de déverrouillage du mécanisme de séparation. En sélectionnant la position correspondante du sélecteur et en appuyant sur le bouton, l'opérateur provoque le déverrouillage puis, après une temporisation, le retour à l'état de vérification, la machine d'assemblage étant réinitialisée à son premier état. Cette possibilité est nécessaire : elle permet de reprendre un assemblage qui s'avérerait incorrect sans avoir à couper l'alimentation du séquenceur ni à revenir aux *ground functions*, désormais inaccessibles.

La stabilisation finale Lorsque toutes les conditions (**JACK_LAUNCH**, assemblage des étages et **JACK_READY**) sont réunies, la fusée entre dans un dernier état d'attente, qui exige que cette situation soit maintenue pendant cinq secondes. Toute rupture d'une condition durant ce délai ramène immédiatement à la boucle de surveillance.

La figure 1.11 illustre la phase d'initialisation du second étage, qui ne diffère de celle du premier que par la liste des actionneurs à mettre en référence et par l'absence de séquence d'assemblage. Bien que cet étage ne possède pas de `JACK_LAUNCH`, la condition correspondante est tout de même évaluée. Lorsque les deux étages sont assemblés, l'action de brancher et débrancher le `JACK_LAUNCH` sur le premier étage se voit répercutée sur le second. Retirer le `JACK_LAUNCH` du premier étage a pour effet d'allumer la led verte sur les deux étages, le brancher de nouveau les éteint.

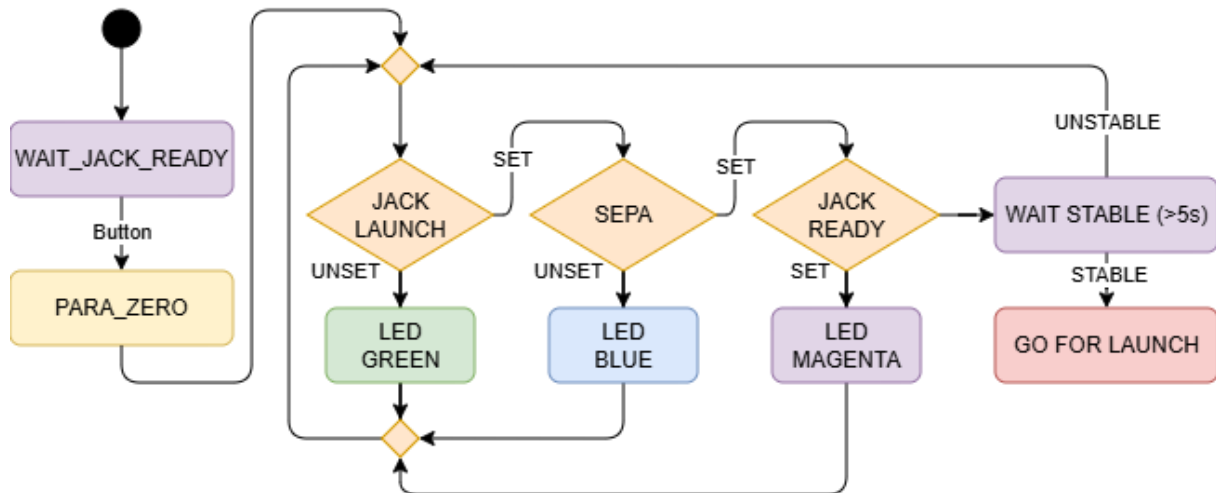


FIGURE 1.11 – Phase d'initialisation du deuxième étage.

Cette étape d'initialisation de la fusée est conçue pour être vécue par l'opérateur plutôt que pour être franchie rapidement. Chaque condition non satisfaite active son propre motif lumineux, dont la couleur identifie ce qui manque : vert pour le `JACK_LAUNCH`, bleu pour l'assemblage des étages, magenta pour le `JACK_READY`. Le jaune, employé lors des étapes précédentes, signale quant à lui une opération d'actionneur en cours. D'un simple coup d'œil, et sans instrument, l'opérateur sait donc ce que le séquenceur attend de lui : une LED verte signifie que le `JACK_LAUNCH` n'est pas inséré alors qu'il devrait l'être, etc. Les figures 1.12 à 1.17 illustrent cette séquence d'initialisation, avec les motifs lumineux correspondants.

À la mise sous tension, les deux étages reposent séparément sur la rampe. Chacun attend l'insertion de son `JACK_READY` et l'annonce par un motif magenta.

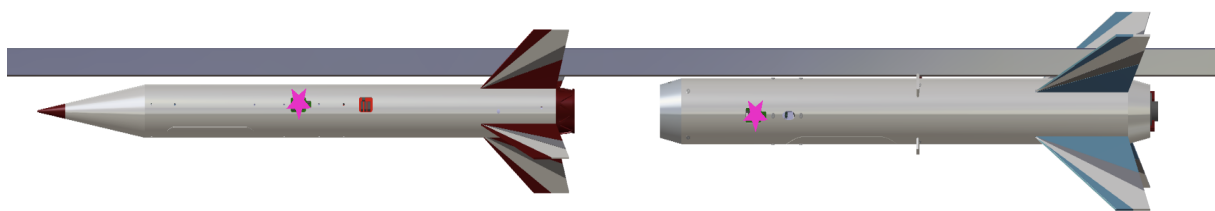


FIGURE 1.12 – Etages non assemblés en rampe, indicateurs magenta sur les deux étages.

Les jacks insérés, l'opérateur valide successivement la mise en référence de chaque actionneur. Le motif jaune signale l'opération en cours : trois étapes sur le premier étage, une seule sur le second.

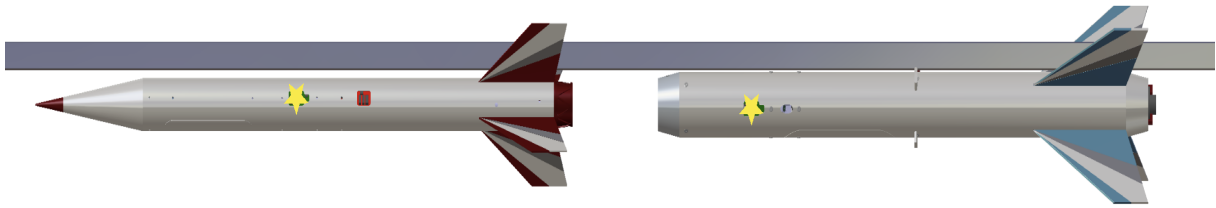


FIGURE 1.13 – Etages non assemblés en rampe, indicateurs jaunes sur les deux étages.

Les actionneurs référencés, les deux étages entrent dans leur boucle de surveillance. Il leur manque alors les deux mêmes conditions : le **JACK_LAUNCH** n'est pas inséré et les étages ne sont pas assemblés. Les motifs vert et bleu alternent donc sur les deux cartes.

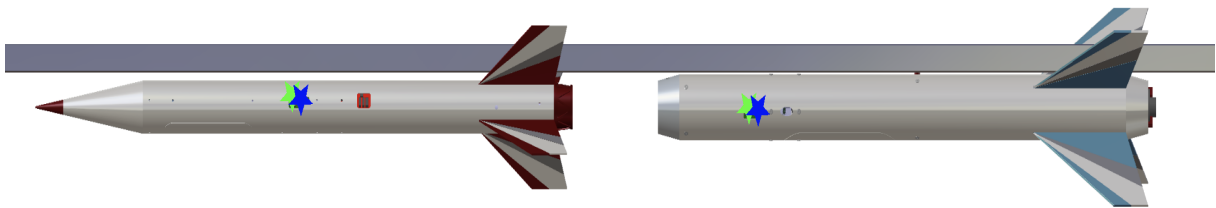


FIGURE 1.14 – Etages non assemblés en rampe, indicateurs verts et bleus sur les deux étages.

L'insertion du **JACK_LAUNCH** sur le premier étage éteint son motif vert. Le second étage, encore désolidarisé, continue en revanche de l'afficher : le signal ne lui parvient que par le couplage mécanique, qui n'est pas encore établi. Les deux étages n'affichent donc temporairement pas la même chose, ce qui est attendu.

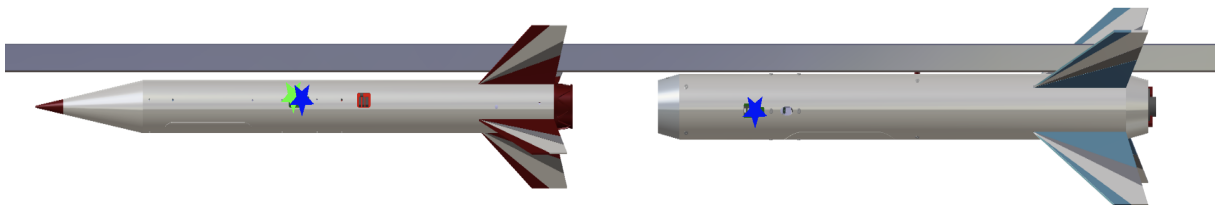


FIGURE 1.15 – Etages non assemblés en rampe, indicateurs bleus sur les deux étages, vert également sur le 2e étage.

Les opérateurs emboîtent alors les deux étages. Le premier détecte le contact, le confirme et commande le verrouillage : les motifs bleus s'éteignent de part et d'autre. Le second étage reçoit du même coup l'état du **JACK_LAUNCH** et éteint son motif vert. Ne subsiste que le magenta, qui réclame cette fois le retrait du **JACK_READY**, dernier geste avant l'évacuation du pas de tir.

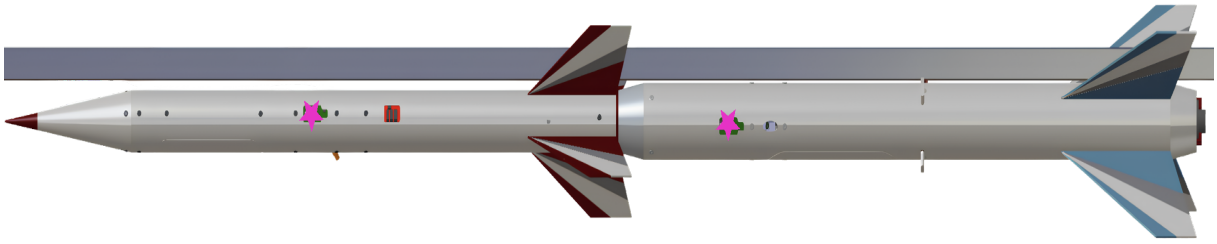


FIGURE 1.16 – Etages assemblés en rampe, indicateur magenta sur les deux étages.

Les JACK_READY retirés et l'ensemble des conditions maintenues pendant cinq secondes, les deux étages basculent en phase de vol et l'annoncent par un motif rouge. La fusée est prête au décollage.

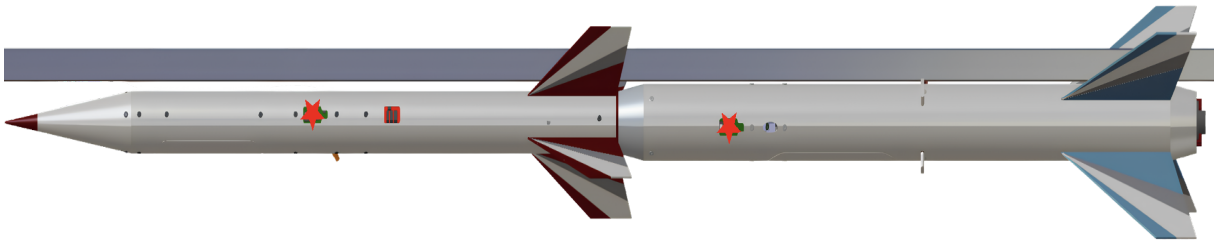


FIGURE 1.17 – Etages assemblés en rampe, indicateur rouge sur les deux étages.

1.2.2.3 Phase de vol

1.2.3 Bibliothèques logicielles du séquenceur

Cette section présente quelques bibliothèques logicielles développées pour le projet et utilisées par le séquenceur. Elles sont détaillées ici pour leur intérêt pédagogique et pour la compréhension du niveau d'abstraction atteint par le programme du séquenceur.

1.2.3.1 Contrôle des actionneurs

Les organes mécaniques de la fusée (aérofreins, trappes de parachute, système de séparation) sont tous mus par des servomoteurs Feetech STS3215. Leur commande repose sur deux bibliothèques logicielles :

- le pilote STS, qui implémente le protocole série du servomoteur ;
- l'utilitaire **actuator**, qui crée une couche d'abstraction du système mécanique dans lequel il se place.

Ce dernier permet de gérer les actionneurs de manière plus abstraite, en permettant de désigner une liste de positions pour chaque système, de passer d'une position à une autre simplement et de pouvoir effectuer une calibration en trouvant les points de référence de chaque système mécanique. La figure 1.18 représente l'articulation de ces deux couches.

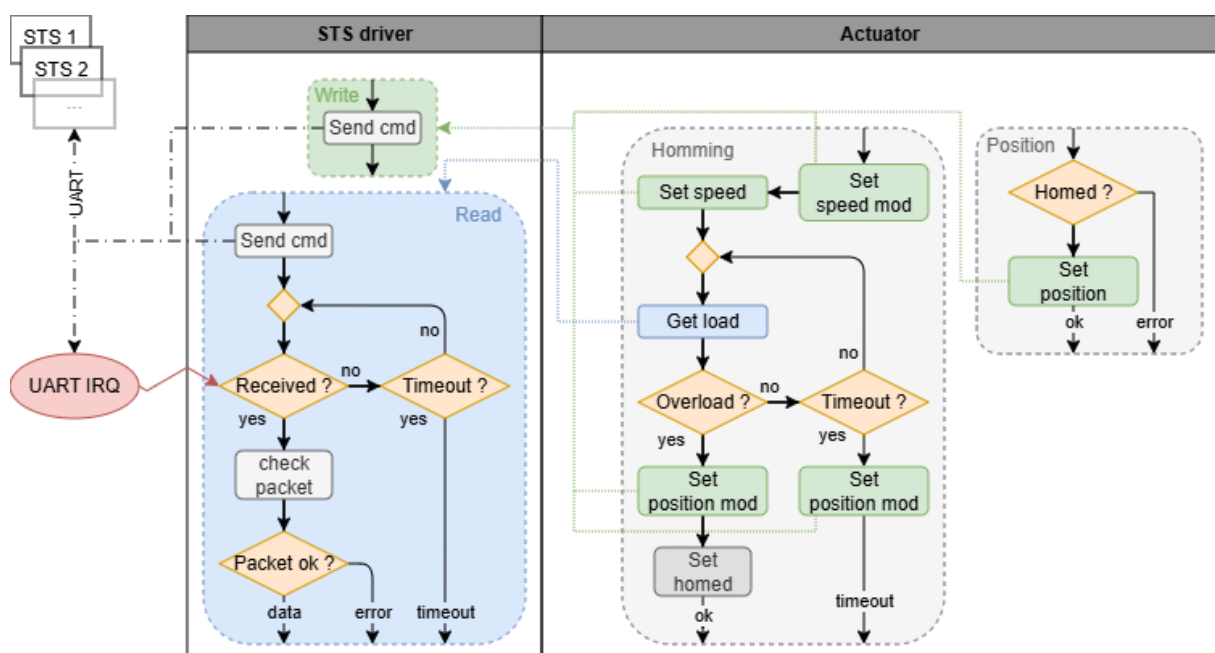


FIGURE 1.18 – Articulation du pilote STS et de l'utilitaire **actuator**

Un bus série partagé et protocole de communication Comme dit précédemment en partie ??, les servomoteurs utilisés dans le projet communiquent sur un bus série partagé UART Half-Duplex.

Le protocole de communication est propriétaire et documenté par le fabricant. Les servomoteurs sont des composants esclaves qui agissent et répondent aux commandes envoyées par le microcontrôleur maître. Le protocole se découpe en deux types de trames :

- les trames d'instruction, qui contiennent les ordres envoyés par le microcontrôleur maître vers le servomoteur esclave ;
- les trames de réponse, qui contiennent les réponses du servomoteur esclave vers le microcontrôleur maître.

Trames d'instruction		Trames de réponse	
Taille	Nom	Taille	Nom
2	Préambule	2	Préambule
1	Identifiant	1	Identifiant
1	Longueur	1	Longueur
1	Instruction	1	Code d'erreur
N	Paramètres	N	Paramètres
1	checksum	1	checksum

TABLE 2 – Types de trames dans le protocole série du servomoteur Feetech STS3215

Le tableau 2 résume les différents types de trames et leur rôle dans le protocole.

Read / Write La figure distingue deux chemins d'exécution dans le pilote. *Write* Consiste à l'envoi d'une trame d'instruction vers le servomoteur sans attente de réponse. Toute opération sur un servomoteur se rapporte à l'écriture d'une valeur dans un registre de celui-ci. *Read* cependant nécessite deux actions : l'envoi d'une trame d'instruction vers le servomoteur, puis la lecture de la trame de réponse. Lorsque le pilote est sollicité pour lire un registre, il envoie la trame d'instruction correspondante, puis attend la réponse du servomoteur via une interruption matérielle sur le bus série concerné. Le pilote vérifie ensuite la validité de la trame reçue et en extrait la valeur du registre demandé. Si la trame n'est pas valide, le pilote signale une erreur. Egalement, si aucune réponse n'est reçue dans un délai fixé, le pilote signale un timeout.

L'aspect partagé du bus série est géré par le pilote d'une manière très simplifiée mais robuste, efficace et suffisante pour le projet. Le logiciel du séquenceur ne s'adresse qu'à un servomoteur à la fois. Puisque les servomoteurs agissent comme des esclaves, il n'y a pas de risque de collision sur le bus série. Lors d'une opération de lecture, le pilote attend la réponse provenant nécessairement du dernier servomoteur sollicité. Le champ d'identifiant dans la trame de réponse permet de vérifier que la réponse provient bien du bon servomoteur.

L'abstraction de ce protocole et la gestion des mécanismes de *Read / Write* sont encapsulés dans le pilote STS.

L'abstraction d'un actionneur

L'utilitaire `actuator` associe à chaque servomoteur une configuration décrivant l'organe qu'il motorise, et conserve l'état de cet organe :

La configuration énumère les positions utiles d'un actionneur, exprimées en degrés, et désigne parmi elles celle qui correspond à sa butée mécanique. Le reste du programme n'a donc jamais à manipuler d'angle : il demande une position par son indice, et la conversion reste interne à la bibliothèque.

```
// .../utils/actuator.h

typedef struct {
    float    homing_speed_rpm;
    uint8_t  homing_calibration_idx;
    float    positions_deg[256];
    uint8_t  num_positions;
} Actuator_Config_t;

typedef struct Actuator_t {
    STS_Servo_t    *servo;
    Actuator_Config_t  config;
    bool            is_homed;
    uint8_t         current_position;
    ...
} Actuator_t;
```

Mise en référence Les positions n'ont de sens qu'une fois le repère interne du servomoteur mis en correspondance avec le repère mécanique du système mécanique qu'il contrôle. Les servomoteurs utilisés dans le projet disposant d'un suivi du couple exercé, les points de référence les plus simples à détecter sont les butées mécaniques de chaque actionneur. La mise en référence consiste donc à déplacer le servomoteur jusqu'à ce qu'il atteigne sa butée, puis à marquer cette position comme repère.

La procédure, représentée sur la figure 1.18, bascule le servomoteur en commande de vitesse, le déplace à vitesse réduite en direction de la butée, puis surveille sa charge jusqu'à détecter la surcharge caractéristique du blocage. Le servomoteur est alors calibré, repassé en commande de position, et l'actionneur est marqué comme référencé.

Cette opération dure plusieurs secondes et ne peut donc pas immobiliser la boucle. Non représenté dans la figure par souci de simplicité, elle est découpée en étapes internes, chaque appel faisant progresser la procédure d'un pas avant de rendre la main (le même motif que celui employé pour les fonctions `sol` en section 1.2.2.1). Si la procédure de mise en référence dépasse un délai fixé, elle est abandonnée et l'actionneur n'est pas marqué comme référencé. Toute commande de position ultérieure sera alors refusée, conformément à la vérification effectuée en début de chemin de commande. Cela permet d'éviter d'ordonner un mouvement à un actionneur qui n'aurait pas été correctement calibré, ce qui pourrait conduire à une collision mécanique et potentiellement à un endommagement de la fusée. Les systèmes mécaniques étant coûteux en temps de travail et en argent, il est préférable de prévenir toute erreur de manipulation.

1.2.3.2 Calcul d'attitude

Afin de réaliser la mission, le séquenceur doit être capable de calculer l'attitude de la fusée à partir des mesures issues d'une centrale inertielle. La construction de l'ordre de décision d'allumage du propulseur du second étage se découpe en 4 étapes principales :

1. Acquisition des mesures de la centrale inertielle **WT901B**.
2. Filtrage des mesures pour réduire le bruit et les erreurs de mesure.
3. Calcul de l'attitude de la fusée à partir des mesures filtrées.
4. Comparaison de l'attitude calculée avec les valeurs attendues pour déterminer si l'allumage du propulseur est autorisé.

Acquisition des mesures et filtrage L'acquisition des mesures de la centrale inertielle est réalisée par le séquenceur via une liaison série **UART** full-duplex. La **WT901B** est capable de fournir des mesures brutes d'accélération, de vitesse angulaire et de champ magnétique, ainsi que des mesures d'attitude calculées par la centrale elle-même sous forme d'angle d'Euler et de quaternion.

Cependant, une incertitude sur les méthodes de calcul utilisées par la centrale et sur les paramètres de calibration de celle-ci ont conduit les équipes du projet à ne pas se fier à ces mesures d'attitude. Bien qu'il est écrit que la **WT901B** est capable de fournir des données cohérente en milieu dynamique, le risque que la forte poussée générée par le propulseur lors du décollage puis la décélération brutale lors de l'extinction de celui-ci fausse les mesures d'attitude est trop important. En effet, ce genre d'algorithme de calcul se sert de l'accélération pour corriger la dérive des gyromètres en considérant que le module se trouve sous l'effet de la gravité terrestre seul.

Après avoir effectué des tests, il a été constaté que les simples mesures brutes des vitesses angulaires pouvaient suffire à la détermination de l'attitude de la fusée. Cette méthode est sujette à une dérive, mais cette dernière est suffisamment faible sur la durée du vol pour que l'attitude calculée reste cohérente. La chaîne de calcul sera également connue et maîtrisée par les équipes du projet, ce qui permet de s'assurer de la cohérence des résultats.

La chaîne d'acquisition et de filtrage utilise 3 bibliothèques logicielles développées pour le projet :

- le pilote **wt901b** qui implémente le protocole série de la centrale inertielle ;
- l'utilitaire **data_topic** qui permet de gérer les données acquises et de les rendre accessibles à d'autres parties du programme via un système de publication/abonnement ;
- l'utilitaire **iir_filter** qui permet de filtrer les données pour réduire le bruit et les erreurs de mesure.

Lors de la phase d'initialisation, la centrale inertielle est configurée de sorte à ce qu'elle fournisse les seules mesures d'accélération et de vitesse angulaire à une fréquence fixe. A chaque envoi de trame par la **WT901B**, une interruption matérielle est levée sur le microcontrôleur du séquenceur qui remplit une mémoire tampon. A chaque itération de la boucle principale, le pilote **wt901b** est appelé pour lire les mesures de la mémoire tampon, décoder la ou les trames reçues et les publier grâce à l'utilitaire **data_topic**.

L'utilitaire **iir_filter** est ensuite appelé pour filtrer les mesures et réduire le bruit et les erreurs de mesure. Cette bibliothèque permet de créer des filtres numériques de type **IIR** (Infinite Impulse Response) pour filtrer les données. Ici, un filtre passe bas de type *moving average* d'ordre 5 est utilisé pour filtrer chaque axe d'accélération et de vitesse angulaire.

Calcul d'attitude Le calcul d'attitude est un problème qui consiste à trouver un calcul de changement de base entre deux repères orthonormés. L'une des représentation la plus moderne et la plus utilisé en robotique est la représentation quaternionique. C'est celle qui est utilisée dans le projet pour calculer l'attitude de la fusée.

On note $\mathcal{R}_{f,n}$ le repère fixe de la fusée à l'instant t_n et $\mathcal{R}_T$ le repère fixe de la Terre. Le quaternion $q_{f,n}$ représente la rotation du repère $\mathcal{R}_{f,n}$ par rapport au repère $\mathcal{R}_T$. Un vecteur $\overrightarrow{v_{(f),n}}$ exprimé dans le repère de la fusée peut être exprimé dans le repère de la Terre par la relation suivante :

$$\overrightarrow{v_{(T)}} = q_{f,n} \otimes \overrightarrow{v_{(f),n}} \otimes q_{f,n}^{-1} \quad (1)$$

où $\otimes$ est le produit quaternionique et $q_{f,n}^{-1}$ est le quaternion conjugué de $q_{f,n}$.

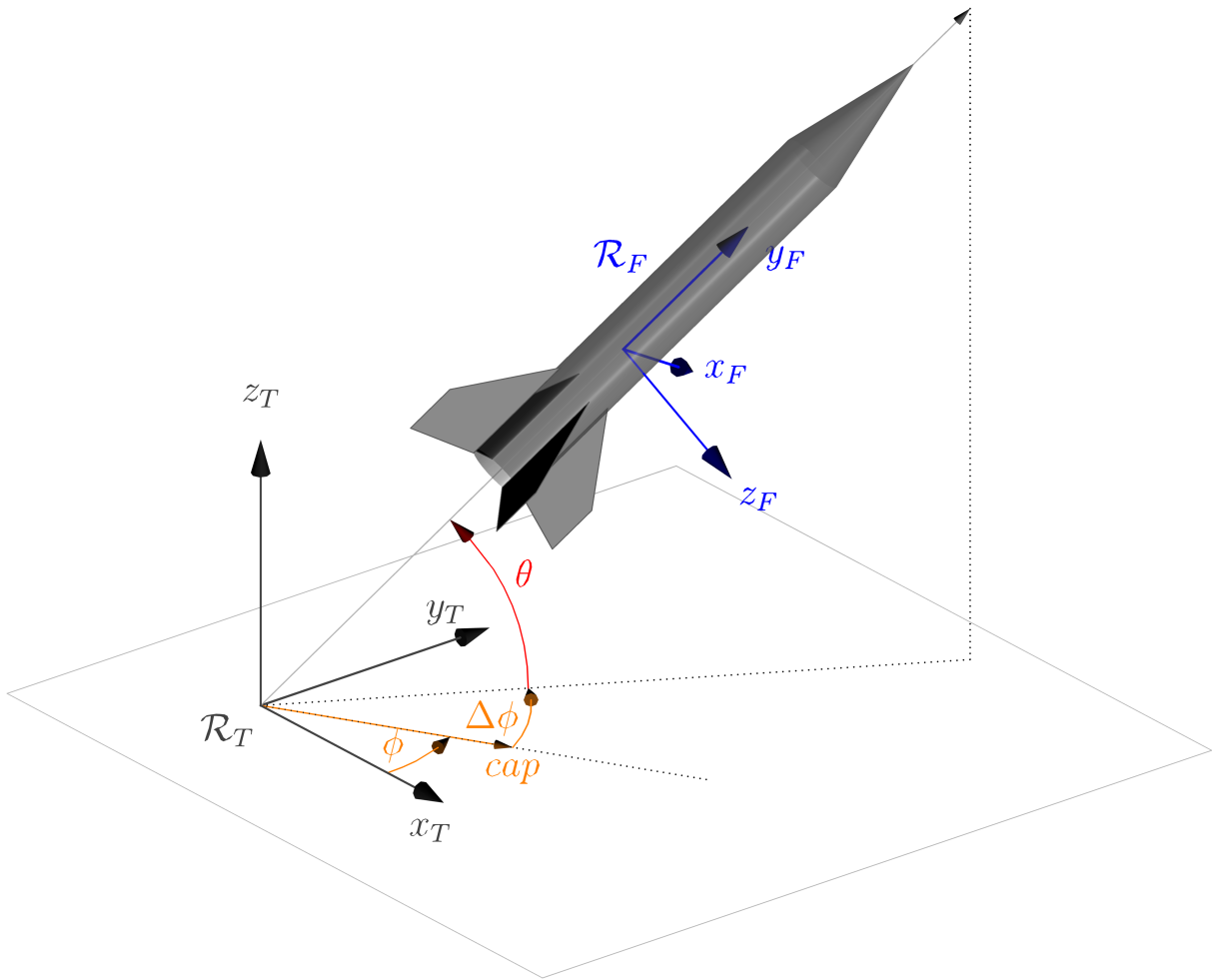


FIGURE 1.19 – Schéma 3D de la fusée et de ces repères.

Connaissant le quaternion $q_{f,n}$ à l'instant t_n , on peut calculer le quaternion à l'instant t_{n+1} en utilisant les mesures de vitesse angulaire $\overrightarrow{\omega_{f,n}} = \begin{pmatrix} \omega_x & \omega_y & \omega_z \end{pmatrix}^T$ fournies par la centrale inertielle. La relation est donnée par :

$$q'_{f,n+1} \approx q_{f,n} + \frac{1}{2} \Delta t \cdot q_{f,n} \otimes \overrightarrow{\omega_{f,n}} \quad (2)$$

$$q_{f,n+1} = \frac{q'_{f,n+1}}{\|q'_{f,n+1}\|} \quad (3)$$

où $\Delta t = t_{n+1} - t_n$ est l'intervalle de temps entre les deux instants. L'erreur associée à cette approximation tendant vers zéro lorsque Δt tend vers zéro.

Cette formule de récurrence permet de calculer l'attitude de la fusée à chaque itération de la boucle principale du séquenceur. Cependant, comme toute formule de récurrence, nécessite une condition initiale. Le quaternion initial $q_{f,0}$ est calculé à partir des mesures d'accélération lorsque la fusée est immobile sur la rampe. La fusée n'étant soumise qu'à la gravité terrestre, l'accélération mesurée par la centrale inertielle donne un vecteur colinéaire à celui de l'axe $\vec{z}_T$ du repère $\mathcal{R}_T$ de la Terre. Pour deux vecteurs normés non colinéaires $\vec{a}$ et $\vec{b}$, le quaternion q qui représente la rotation de $\vec{a}$ vers $\vec{b}$ est donné par :

$$q'_{a \rightarrow b} = \begin{pmatrix} \vec{a} \cdot \vec{b} + 1 \\ \vec{a} \wedge \vec{b} \end{pmatrix} \quad (4)$$

$$q_{a \rightarrow b} = \frac{q'_{a \rightarrow b}}{\|q'_{a \rightarrow b}\|} \quad (5)$$

En notant $\vec{a}_f$ le vecteur d'accélération normé mesuré par la centrale inertielle dans le repère de la fusée, le quaternion initial est donc donné par : $q_{f,0} = q_{\vec{a}_f \rightarrow \vec{z}_T}$.

A partir du quaternion $q_{f,n}$, il est possible de calculer les angles d'élévation θ et d'azimut ϕ de la fusée via des projections de vecteurs dans le repère de la Terre. L'angle d'élévation est l'angle entre l'axe $\vec{z}_T$ et le vecteur longitudinal de la fusée $\vec{y}_{f,n}$. Via la relation 1, on peut calculer le vecteur longitudinal de la fusée exprimé dans le repère de la Terre $\vec{y}_{f,n,(T)}$, puis on calcule l'angle par :

$$\theta_n = \frac{\pi}{2} - \arccos(\vec{y}_{f,n,(T)} \cdot \vec{z}_T) \quad (6)$$

La détermination de l'angle d'azimut nécessite de projeter le vecteur longitudinal de la fusée dans le plan horizontal du repère de la Terre. Le vecteur projeté $\vec{y}_{f,n,(T),proj}$ est donné par :

$$\vec{y}_{f,n,(T),proj}' = \vec{y}_{f,n,(T)} - (\vec{y}_{f,n,(T)} \cdot \vec{z}_T) \vec{z}_T \quad (7)$$

$$\vec{y}_{f,n,(T),proj} = \frac{\vec{y}_{f,n,(T),proj}'}{\|\vec{y}_{f,n,(T),proj}'\|} \quad (8)$$

Représentant le cap de la fusée, nous noterons $\vec{cap}_n = \vec{y}_{f,n,(T),proj}$. L'angle d'azimut, pris entre le vecteur $\vec{x}_T$ représentant la direction Nord et le vecteur $\vec{cap}_n$, n'est pas en soi une information utile à la décision d'allumage du propulseur. Ce qui donne cette décision est la différence entre l'angle d'azimut de la fusée et l'angle d'azimut de la cible. Cette différence notée $\Delta\phi_n$ est l'angle que forme les vecteurs $\vec{cap}_n$ et $\vec{cap}_0$. Similairement à l'angle d'élévation, il est calculé par :

$$\Delta\phi_n = \arccos(\vec{cap}_n \cdot \vec{cap}_0) \quad (9)$$

A chaque itération de la boucle principale, le séquenceur détermine l'attitude de la fusée en calculant $q_{f,n}$. Lorsque la fusée n'a pas encore décollé, le séquenceur calcule $q_{f,0}$ à partir des mesures d'accélération. Il est réécrit à chaque itération jusqu'au décollage. Cela permet au système de se calibrer non pas à partir d'une position initiale arbitraire, mais à partir de la position réelle de la fusée sur la rampe. Dès lors que le décollage est détecté, le séquenceur calcule $q_{f,n}$ à partir de la formule de récurrence. C'est seulement à ce moment qu'une dérive liée à l'intégration des mesures de vitesse angulaire peut apparaître. Cependant, la durée du vol étant relativement courte, cette dérive reste faible.

Toutes les briques mathématiques nécessaires pour la manipulation des vecteurs 3D et des quaternions sont implémentées dans 3 bibliothèques :

- `float3` pour la manipulation de vecteurs 3D, avec notamment les opérations de produit scalaire et vectoriel ;
- `quaternion` pour la manipulation de quaternions, avec notamment les opérations de produit ;
- `quaternion_dynamics` pour la formule de récurrence du calcul d'attitude.

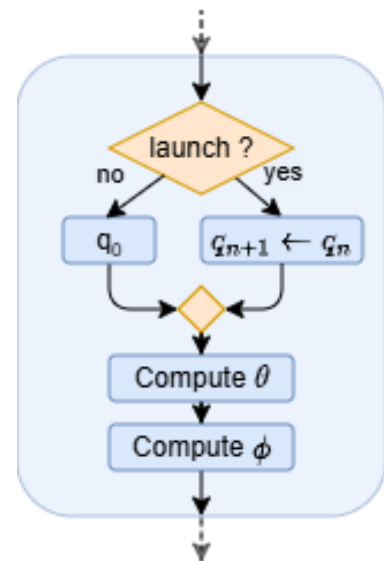


FIGURE 1.20 – Calcul d'attitude

Comparaison avec les valeurs attendues Les valeurs d'attitude étant calculées, le séquenceur doit s'en servir afin de déterminer si l'allumage du propulseur du second étage est autorisé.

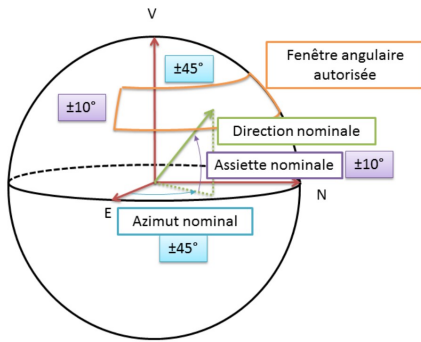


FIGURE 1.21 – Fenêtre d'attitude autorisée pour l'allumage du propulseur du second étage.

Les conditions d'allumage sont les suivantes :

- l'angle d'élévation θ_n doit se trouver dans un intervalle de tolérance de $\pm 10^\circ$ autour de l'angle d'élévation nominal $\theta_{nom,n}$;
- l'angle d'azimut ϕ_n doit se trouver dans un intervalle de tolérance de $\pm 45^\circ$ autour de l'angle d'azimut nominal $\phi_{nom,n}$.

Ces angles nominaux représentent ceux que la fusée doit théoriquement atteindre à l'instant t_n lors d'un vol stable sans vent et sans perturbation. L'angle $\phi_{nom,n}$ est déterminé par la direction choisie par les équipes du CNES lors de l'orientation de la rampe de lancement. L'angle $\theta_{nom,n}$ est lui déterminé via la simulation de vol donnée par le Stabtraj.

A l'aide d'Excel, l'évolution théorique de l'angle d'élévation $\theta_{nom,n}$ a été modélisé en fonction du temps par une équation polynomiale du 3ème degré.

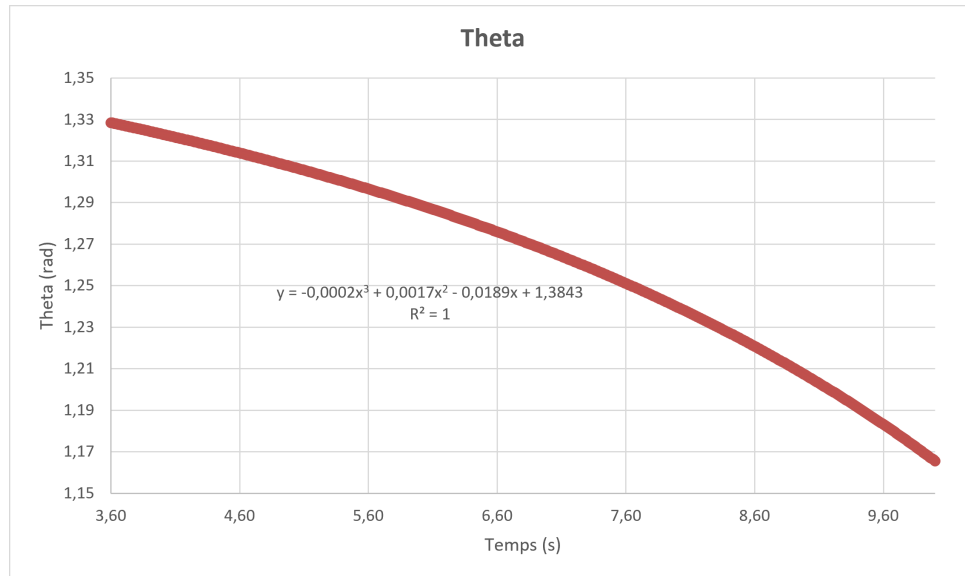


FIGURE 1.22 – Fenêtre d'attitude autorisée pour l'allumage du propulseur du second étage.

La méthode des moindres carrés a été utilisée sur une portion du vol allant de 3.6 s à 10.0 s. Sur cette portion de vol, l'évolution de l'angle d'élévation est modélisé par l'équation polynomiale suivante :

$$\theta_{nom,n} = -0.0002t_n^3 + 0.0017t_n^2 - 0.0189t_n + 1.3843 \quad (10)$$

Le coefficient de détermination R^2 est très proche de 1, ce qui signifie que l'équation polynomiale du 3ème degré est une bonne approximation de l'évolution de l'angle théorique.

A chaque itération, la condition d'allumage est alors :

$$\begin{cases} \|\Delta\theta_n\| = \|\theta_{nom,n} - \theta_n\| \leq 10^\circ \\ \|\Delta\phi_n\| = \|\phi_{nom,n} - \phi_n\| \leq 45^\circ \end{cases} \quad (11)$$

1.2.3.3 Gestion des événements lumineux

1.3 Expérience

1.3.1 Présentation générale

1.3.2 Real-Time Operating System (RTOS)

1.3.3 Gestion des flux de données

1.3.4 Chaine de télémesure

1.3.4.1 Méthode du Time Division Multiplexing

1.3.4.2 Configuration du protocole LORA

1.3.4.3 Configuration du protocole FSK

1.3.5 Protocole de communication avec le séquenceur

1.4 Caméra embarquée

Le logiciel embarqué a pour seule fonction de déclencher l'enregistrement des caméras après la mise sous tension de la carte. La vidéo est ensuite stockée directement par les caméras et n'est pas traitée par l'ESP32 Xiao C3.

1.4.1 Séquence de démarrage

À la mise sous tension, le programme initialise l'ESP32 Xiao C3 et ses interfaces, puis démarre un compteur de 10 min. À l'issue de ce délai, les ordres de démarrage sont envoyés aux caméras. La séquence est donc la suivante :

1. initialisation de la carte ;
2. démarrage du compteur ;
3. expiration du délai de 10 min ;
4. envoi des ordres de démarrage ;
5. début de l'enregistrement.

Après l'expiration du délai, le programme commande la RunCam Nano 2 par un changement d'état de son entrée High/Low. Il envoie simultanément une commande de démarrage par UART aux RunCam Thumb Pro. Le même ordre est reçu par les deux Thumb Pro lorsqu'elles partagent la ligne de réception.

1.4.2 Développement du code

Le protocole de commande UART a été repris à partir du dépôt RunCam Thumb ESP32-C3 Controller[?], puis adapté au fonctionnement de la carte. La temporisation de démarrage est fixée à 10 min.

Les caméras enregistrent les vidéos sur leurs cartes SD et conservent les fichiers même lorsque leur alimentation est coupée. Il n'est donc pas nécessaire de prévoir une commande d'arrêt spécifique en fin de vol. Lorsque les batteries arrivent en fin de décharge, leur BMS coupe l'alimentation afin de les protéger. La carte caméra n'est alors plus alimentée et cesse à son tour d'alimenter les caméras, sans que les vidéos déjà enregistrées soient perdues. La capacité des cartes SD permet de stocker plusieurs heures de vidéo, soit une durée supérieure à l'autonomie des batteries. De plus, les caméras découpent les enregistrements en fichiers de 10 min, ce qui limite les risques de perte d'un enregistrement complet lors de la coupure d'alimentation.

Des essais au sol ont permis de valider les ordres de déclenchement et d'arrêt, ainsi que la récupération des vidéos enregistrées. La réception des commandes par les entrées de la PCB, après mise en forme par les triggers de Schmitt, a également été vérifiée. Ces essais ont confirmé le fonctionnement de la chaîne de commande et de l'enregistrement vidéo.

La commande par les entrées n'a toutefois pas été utilisée pendant le vol. La configuration retenue reposait uniquement sur le compteur de 10 min intégré au logiciel de la carte caméra.

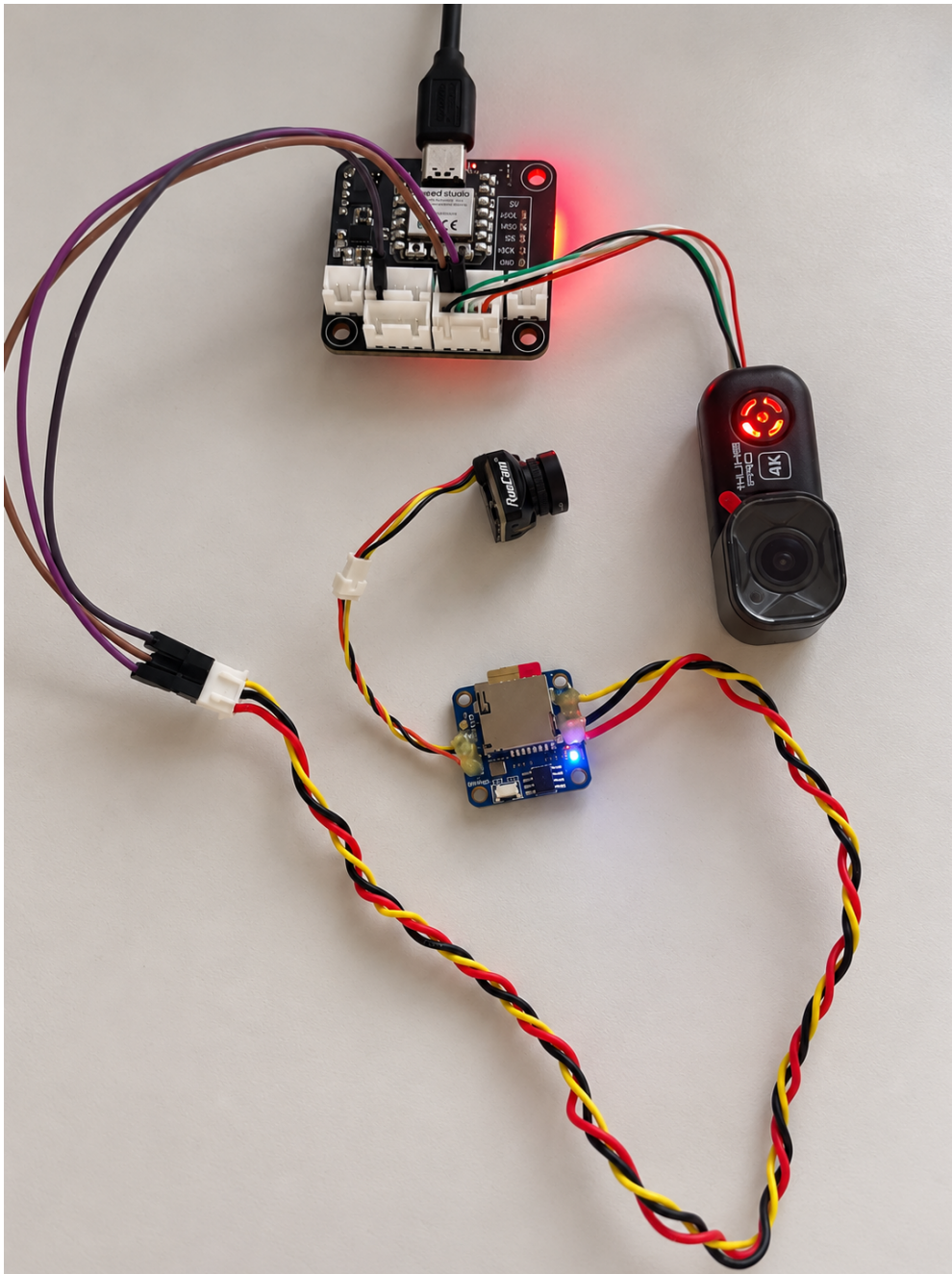


FIGURE 1.23 – Essais au sol du système de caméras.

Table des figures

1.1	Configuration d'un bus UART avec STM32CubeMX	3
1.2	Configuration d'horloges avec STM32CubeMX	4
1.3	Arborescence d'un dépôt logiciel type utilisé pour le séquenceur et l'expérience APEX	7
1.4	Chaîne de compilation d'un dépôt logiciel type du projet	7
1.5	Boucle séquentielle (superloop)	8
1.6	Système d'exploitation temps réel (RTOS)	8
1.7	Architecture générale du programme du séquenceur	9
1.8	Sélection et exécution d'une fonction sol	11
1.9	Phase d'initialisation du premier étage.	13
1.10	Zoom sur la séquence d'assemblage des étages.	14
1.11	Phase d'initialisation du deuxième étage.	15
1.12	Etages non assemblés en rampe, indicateurs magenta sur les deux étages.	15
1.13	Etages non assemblés en rampe, indicateurs jaunes sur les deux étages.	16
1.14	Etages non assemblés en rampe, indicateurs verts et bleus sur les deux étages.	16
1.15	Etages non assemblés en rampe, indicateurs bleus sur les deux étages, vert également sur le 2e étage.	16
1.16	Etages assemblés en rampe, indicateur magenta sur les deux étages.	17
1.17	Etages assemblés en rampe, indicateur rouge sur les deux étages.	17
1.18	Articulation du pilote STS et de l'utilitaire actuator	19
1.19	Schéma 3D de la fusée et de ces repères.	22
1.20	Calcul d'attitude	23
1.21	Fenêtre d'attitude autorisée pour l'allumage du propulseur du second étage.	24
1.22	Fenêtre d'attitude autorisée pour l'allumage du propulseur du second étage.	24
1.23	Essais au sol du système de caméras.	27

Liste des tableaux

1	Fonctions sol disponibles sur chaque étage	12
2	Types de trames dans le protocole série du servomoteur Feetech STS3215	19

Références

- [1] Planète Sciences et le CNES, “Logiciel de simulation de trajectoire Stabtraj.” <https://www.planete-sciences.org/espace/Documentation-technique/Trajectoire>, 2024.

Glossaire

C

C'Space Campagne nationale de lancements de fusées expérimentales organisée par le CNES et l'association Planète Sciences. Elle est exclusivement réservée aux étudiants des écoles d'ingénieurs et universités françaises. Cette campagne se déroule chaque année dans le camp de Ger à Tarbes dans les Hautes-Pyrénées au début du mois de juillet.. 30

Centre National d'Études Spatiales Agence spatiale française, responsable des programmes spatiaux civils en France.. 30

L

Long Range Technologie de communication sans fil basse consommation et longue portée, utilisée pour la transmission de données sur de grandes distances avec une faible consommation d'énergie, souvent employée dans les applications de l'Internet des Objets (Internet of Things) (IoT).. 30

P

Planète Sciences Association française de promotion des sciences et des techniques auprès des jeunes, organisatrice de la campagne C'Space en collaboration avec le CNES.. 29, 30

R

Real-Time Operating System Système d'exploitation conçu pour traiter les tâches dans des délais stricts, souvent utilisé dans les applications critiques où le temps de réponse est essentiel.. 2, 25, 30

S

Stabtraj Outil de dimensionnement et de simulation de trajectoire de fusée développé par l'association Planète Sciences en collaboration avec le CNES. Il permet d'analyser la stabilité, la performance et la trajectoire des fusées amateur. [1]. 24, 29

Acronymes

A

API Application Programming Interface. 4

C

CNES Centre National d'Études Spatiales. 24, 29, 30

F

FSK Frequency-Shift Keying. 2, 25

H

HAL Hardware Abstraction Layer. 2, 3, 4, 5

I

I2C Inter-Integrated Circuit. 3

IHF Interface Homme Fusée. 9, 11

IoT Internet des Objets (Internet of Things). 30

L

LORA Long Range. 2, 25

R

RTOS Real-Time Operating System. 2, 8, 25

S

SPI Serial Peripheral Interface. 5

T

TDM Time Division Multiplexing. 2, 25

U

UART Universal Asynchronous Receiver-Transmitter. 3

USB Universal Serial Bus. 12